1 Lesson 28-10-26

Last lesson we talked about *serial* and *parallel* interconnects. We hinted at the fact that parallel is more performant (as we can transmit entire words per clock cycle).

1.0.1 Serial vs. parallel

The thing is that, performance of serial transmission is not affected by skew as the transmitted signal actually always shows a delay with some skew: but data and clock, being on the same wire, are affected in the same way. As long as the phase delay between data and clock is maintained, if the skew is significantly higher than setup and hold times, the clock frequency of a serial transmission can be significantly higher than the clock frequency of a parallel transmission. This means that serial can have higher performance than parallel when:

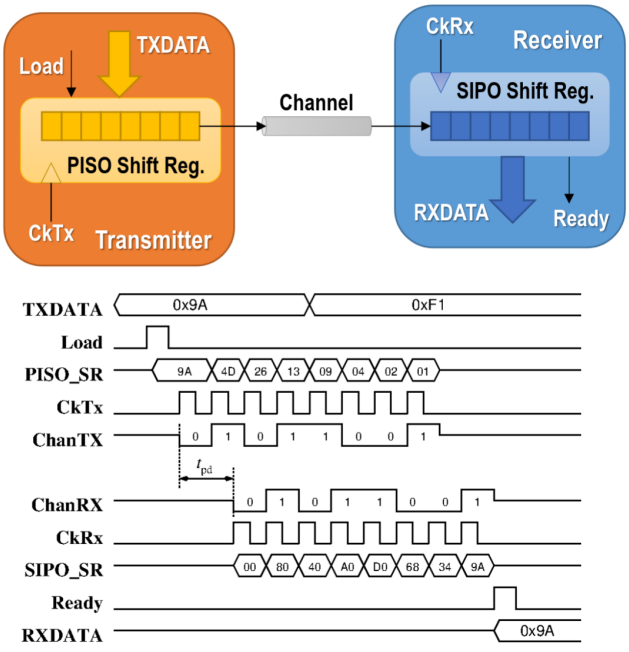
$$f_{\text{ckser}} > n \cdot f_{\text{ckpar}}$$

This means that today most high performance interconnections are *serial*, especially for long wires (high skew). Parallel busses survive outside ICs only for *memory interconnections*, which is high performance, and needs extreme care in design, and low performance communications with short interconnections.

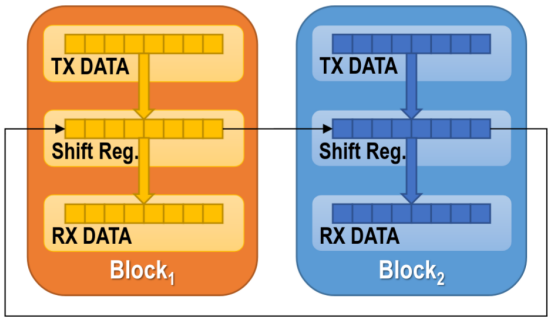
1.1 Basic serial transmission

Let's start looking at the structure of a serial **transmitter** sending to a serial **receiver**.

- The signal **transmitted** is a sequence of symbols, loaded into a register. Each symbol carries a single bit or multiple bits. Data at the transmitter is copied into a **PISO SR** (*Parallel Input, Serial Output Shift Register*), with the load signal activated.
- Data is then transferred from the transmitter to the **receiver** through a communication channel. The receiver has a **SIPO SR** (*Serial Input, Parallel Output Shift Register*). Th transmit and receive clocks are synchronized, so that they maintain a known phase relationship. When data at receiver is ready, a data valid signal is raised.



To enable full duplex transmission we may be tempted to have a transmitter and a receiver on each side, but we can get away with connecting the shift registers in a loop, such as:



The pipeline each block (receiver/transmitter) follows is as follows:

1. Data to be transferred is in the TXDATA register of each block, respectively, and data from TXDATA is transferred to the Shift Register. Blocks make this transfer without synchronizing.
2. Shift Registers of both blocks are clocked synchronously. Data is shifted bit by bit from one shift register to the other. After n clock cycles (for n bit shift registers), data has been transferred completely;
3. The contents of each Shift Register are transferred to the RXDATA register. Blocks make this last transfer without synchronizing.

1.1.1 Clock Sync.

Proper clock transmission allows **bit** synchronization, which guarantees correct sampling of single bits (when loading in the receiver shift register). This needs to synchronzie the clock. Higher protocol levels allow *character* (endianess, ...) and *message* synchronization message delimiters, ...).

The clock can be generated at the transmitter or receiver side, then sent to receiver or transmitter on a parallel line with respect to data. This is limited by skew, but at least (as we said) the skew is mostly the same between clock and data.

If sent by the receiver, the clock is also limited by transmission delay.

Clock can also be *embedded* (*Clock Data Recovery*, **CDR**): generated by transmitter, encoded in the data (presence of transitions), extracted by the receiver. This means data changes periodically (define maximum interval between transitions), and continuously (to allow synchronization).

Lastly, we talk about *isofrequential* clocks: in this case the transmitter and receiver have separate free running isofrequential clocks. Of course these need to be periodically synchronized.

1.2 UART serial transmission

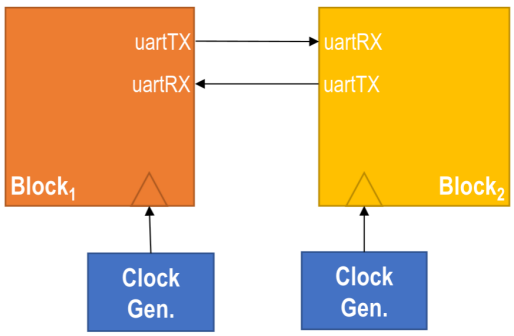
Let's now talk about a real example of serial communications, which is **UART** (*Universal Asynchronous Receiver Transmitter*).

UART consists of a point-to-Point serial communication, with:

- Isofrequential independent clocks;

- Two serial wires;
- One for transmission from Block1 to Block2.
- One for transmission from Block2 to Block1.

Ground is of course included but not shown.



As with most isofrequential systems, there's a periodic synchronization of the clocs. After the synchronization the clocks run free.

1.2.1 Character format

Transmission in UART is character based. Each transmitted bit lasts one clock cycle, and each transmission consists of a *frame*, which contains a character of n bits.

A frame has a defined format:

- START BIT (1 bit);
- Character (n bits, LSB first);
- PARITY BIT (1 bit, optional, might be absent);
- STOP BIT (1 or 2 bits).

Synchronization between clocks happens at the START BIT only.

We also need to talk about frame synchronization.

In the UART transmission format, an UART frame can start at any time. Each bit nominally lasts one clock cycle, so:

$$t_{\text{bit}} = \frac{1}{f_{\text{ck}}}$$

Character and parity bits are transmitted using standard NRZ encoding, so there are no guaranteed transitions. How do we guarantee a transition in each frame (so that the receiver can notice when a transmission starts)?

The UART line is kept at logic 1 when idle, and STOP BITs are always at logic 1. The START BIT is always a logic 0. This means that either after an idle phase, or following the previous frame, the START BIT of a new frame always causes a 1 to 0 transition.

This START BIT transition allows synchronizing clocks at the beginning of a frame. There is always at least a 0 to 1 transition (the first high data bit), as well, but its position is not known a priori, so it cannot be used for synchronization.

1.2.2 Transmission parameters

Transmitter and receiver should also agree on some parameters.

- **Baud rate** (symbols per second): it is the nominal clock frequency f_{ck} of the independent clocks. It corresponds to bps (bits per second) in this context, but symbols in general may represent more than one bit;
- **Number** n of bits in a character;
- Presence and kind of **parity** bits (as we said the parity bit can be not present);
- Number of **stop** bits (either 1 of 2).

Note that the configuration of these parameters is outside of the UART protocol.

1.2.3 Overhead

We saw that UART transmits some bits (START, STOP, eventual parity...) which are not directty useful by themselves. These are **overheads**. The minimum overhead is given by the STOP - START transmission on frame start, so 2 bits.

We have that the maximum transmission rate depends on both:

- **Baud Rate**: The higher the baud rate, the higher the performance. There are f_{ck} bit times in a second;
- **Overhead**: START BIT, Parity and STOP BITs do not carry information. AS we said, the minimum overhead is 2 bits (START BIT, 1 STOP BIT). The maximum overhead, on the other hand, is 4 bits (START BIT, Parity, 2 STOP BITs.

The equation for the ffective useful bit rate is:

$$\text{bit rate} = \frac{n}{n + o_h} f_{\text{ck}}$$

Increasing f_{ck} and n might increase performance, but there are limits: sampling is performed within a certain window where data is stable.

The window is between two active clock edges, where data potentially changes. The window is defined by a set of parameters:

- Voltage levels (V_{OL}, V_{IL}, V_{OH}, V_{IH});
- Noise (NML, NMH);
- Timing (setup t_{su}, hold t_h, rising t_r and falling t_f, slew rate, bandwidth).

Increasing the clock frequency makes the window smaller. In addition to this clock drifting moves the sampling point outside the window after a certain number of bits without synchronization. Once windows parameters are set, maximum n and f_{ck} can be computed

1.2.4 UART performance

Let's talk UART Performance.

For **long interconnections**, performance is limited by attenuation, distortion, noise. These affect the vertical axis in the *eye diagram* (see later).

Performance is also limited by unpredictable delays These affect the horizontal axis in the eye diagram (see later). It is improved by large voltage swings.

For **short interconnections**, erformance is limited by clock jitter and tolerances. These affect the horizontal axis in the eye diagram (see later).

Note that interconnection length and speed are conflicting parameters: we can have a short and fast link, or a long and slow link.

1.3 Eye diagrams

An **eye diagram** defines the sampling window via signal waveforms at the receiver. It is built considering all possible transitions between transmitted bits, and showing all transitions superimposed on the same graph.

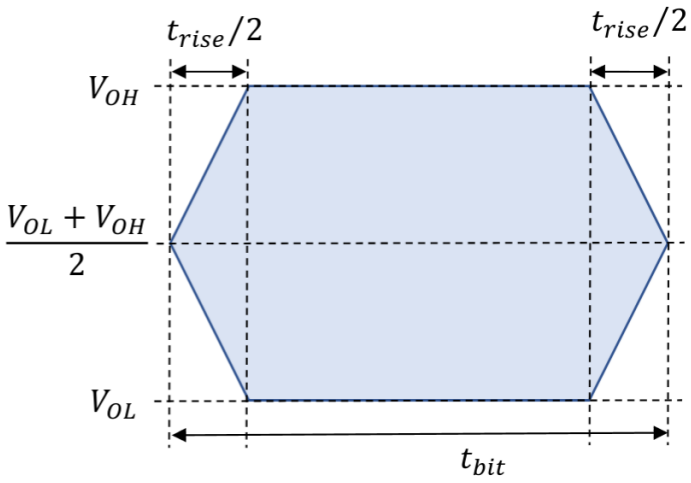
Let's draw an eye diagram ourselves. Let's start with a time interval equal to t_{bit} and a voltage range between V_{OL} and V_{OH} for the given technology, out of the typical 0 to V_{cc} range. Let's now assume for simplicity that $t_{\text{rise}} = t_{\text{fall}}$.

We start with ramps at the left side which start at $\frac{V_{OL}+V_{OH}}{2}$ and go toward V_{OL} and V_{OH} respectively, in time t_{rise} (the down ramps at the rights side will be mirrors of this if $t_{\text{fall}} = t_{\text{rise}}$). For large signals, we can use the **slew rate** S_R:

$$t_{\text{rise}} = \frac{V_{OH} - V_{OL}}{S_R}$$

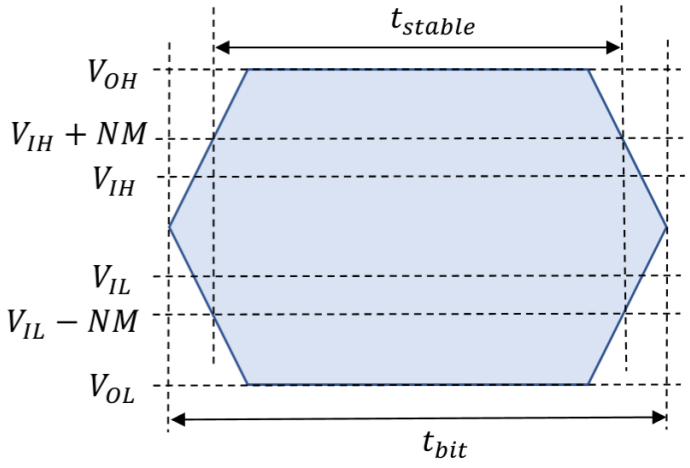
We then do the same thing on the right side, ensuring the total time away from $\frac{V_{OL}+V_{OH}}{2}$ is t_{bit}.

The result is shown below.



Let's note the noise margins in the graph and see where, in both sides (up and down) the signal is considered stable.

- We identify the value as high for a time t_{stable} if the graph is above $V_{IH} + NM$.
- We identify the value as low for a time t_{stable} if the graph is below $v_{IL} - NM$.

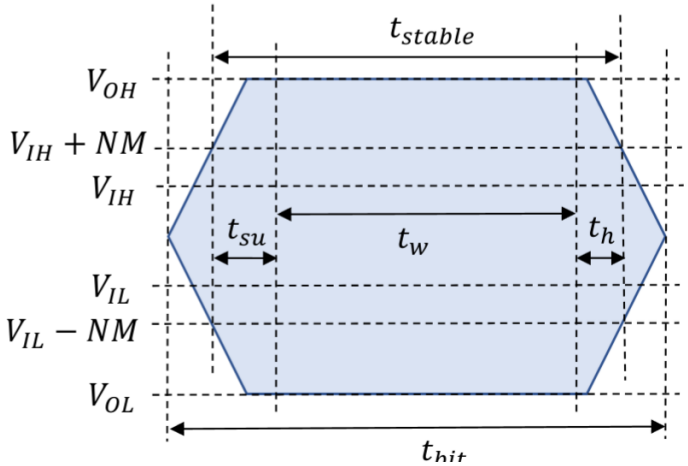


This is still not enough, as the value has to be stable for enough time for the register to setup (t_su) and to hold (t_h). This gives us the actual sampling window t_w, where we always have:

$$t_w < t_{\text{stable}} < t_{\text{bit}}$$

Sampling inside the sampling window t_w is always correct as:

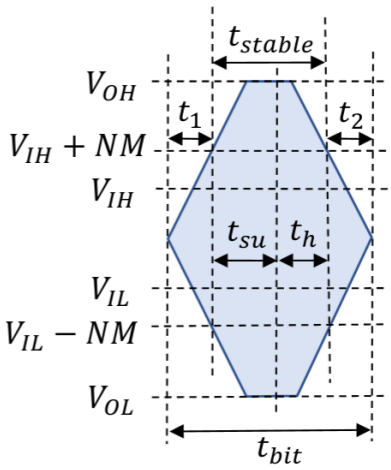
- The signal is stable;
- The signal has proper voltage levels;
- Setup and hold time are satisfied.



Now, immediately after synchronization, the receiver determines the best sampling instant. We assume it is at the center of the sampling window. The main isse is that clock drift eventually might get the sampling instant outside the sampling window.

This means we have 2 limitations:

- *Increasing f_{ck} decreases the t_w window, making the clock drift out faster.*
  With perfect isofrequential clocks, we synchronize clocks on START BIT and always assume sampling happens within the sampling window for all successive bits. So there is no limit to the number n of consecutive bits, but here is still a limit to the maximum clock frequency.
  With real rising and falling edge, we compute the window t_w as:



$$t_1 = \left(\frac{V_{IH} + NM}{V_{OH} - V_{OL}} - \frac{1}{2} \frac{V_{OH} + V_{OL}}{V_{OH} - V_{OL}} \right) t_{\text{rise}}, \quad t_2 = t_1$$
$$f_{\text{ck}} = \frac{1}{t_{su} + t_h + t_1 + t_2} = \frac{1}{t_{su} + t_h + 2t_1}$$

The maximum clock frequency limits the maximum performance.

- *Increasing n makes the clock to drift out towards the end of the transmission.*

  Let's assume nominally isofrequential clocks with tolerances. This means there's a relative error ε (in percentage) from the nominal clock frequency at the receiver:

$$\begin{cases} f_{\text{ck}} &= f_{\text{nominal}} \pm \delta_f \\ t_{\text{bit}} &= t_{\text{nominal}} \pm \delta_t \end{cases} \rightarrow \begin{cases} f_{\text{ck}} &= f_{\text{nominal}} \pm \varepsilon_f \% \\ t_{\text{bit}} &= t_{\text{nominal}} \pm \varepsilon_t \% \end{cases}$$

  With relative errors:

$$\varepsilon_f = \frac{\delta_f}{f_{\text{nominal}}}, \quad \varepsilon_t = \frac{\delta_t}{t_{\text{nominal}}}$$

  Because of this relative error, the sampling point from one bit to another differs by an amount:

$$\Delta t = t_{\text{bit}}^{RX} - t_{\text{bit}}^{TX}$$

  In the worst case:

$$\Delta t = t_{\text{nominal}} \cdot 2\varepsilon$$

  Considering equal tolerances at transmitter and receiver, but in opposite directions.

  As we were saying, drift may cause sampling outside the sampling window after a certain number of consecutive bits without re-synchronization.

In the eye diagram there is one degree of freedom: once we determine (estimate or measure) all parameters, we keep the clock frequency f_{ck} and the number n of consecutive bits unknown. Fixed the clock frequency and its tolerance, we compute the maximum number of consecutive bits: t_{bit} is known and equal to t_{nominal} when drawing the eye diagram.

We can also do the reverse: fix the number of consecutive bits and the clock tolerance, compute the maximum clock frequency. This leaves t_{bit} as an unknown when drawing the eye diagram.